01 — Architecture & System Overview

Why JOCKY Exists

Anti-virus software detects malicious tools using two primary methods:

Method 1 — Static signature matching. AV maintains a database of known-bad file hashes. If the SHA-256 of your binary matches a known threat, it is blocked before it even runs. The weakness: this only catches tools that have been seen before, in exactly the same byte-for-byte form.

Method 2 — Static string/pattern scanning. AV parses the binary and looks for suspicious strings (`cmd.exe`, `HKEY_LOCAL_MACHINE`, `VirtualAllocEx`, `EnumProcesses`) or code patterns (function call sequences that resemble shellcode loaders). Many tools are caught purely by recognisable strings in the compiled binary.

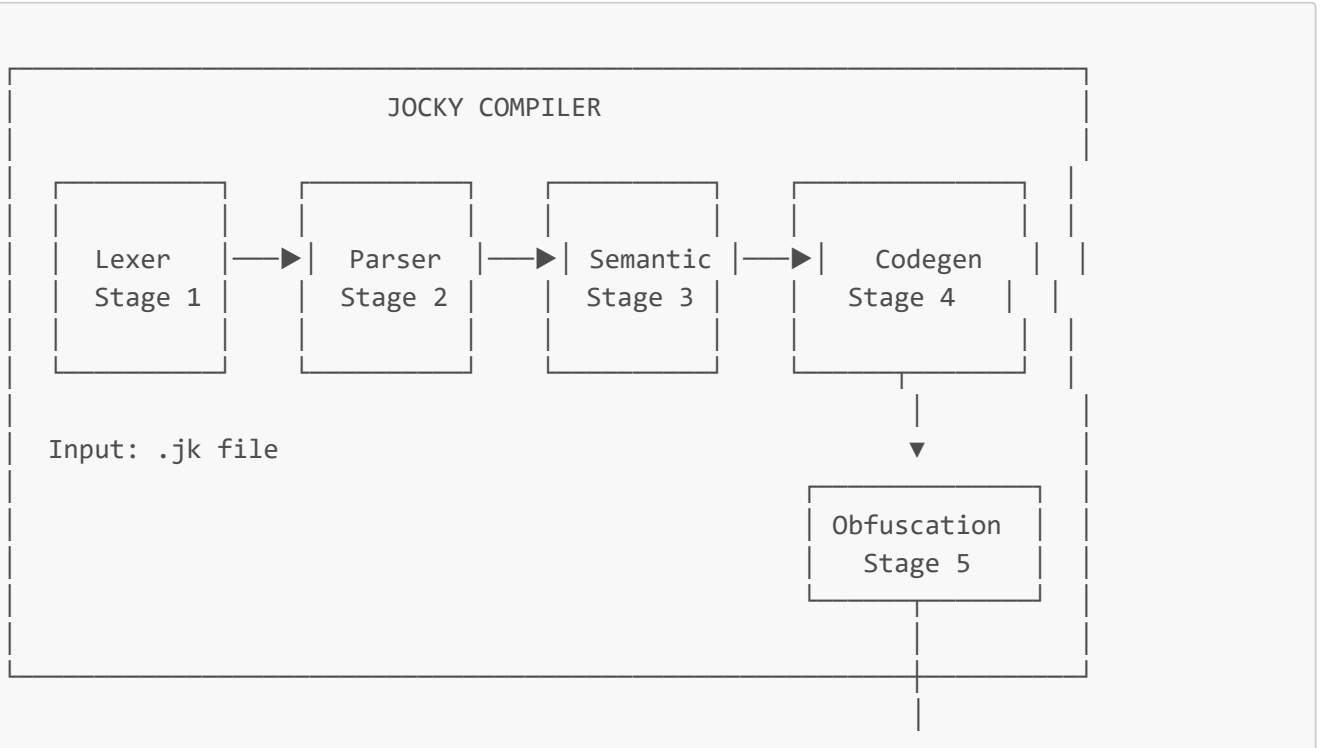
Method 3 — Script parser detection. When a Python or PowerShell script is submitted to an AV engine, the engine parses the script's syntax. It recognises `import os`, `subprocess.Popen`, `TerminateProcess`, etc. Unknown scripting languages produce parse errors in AV engines — they cannot reason about code they cannot parse.

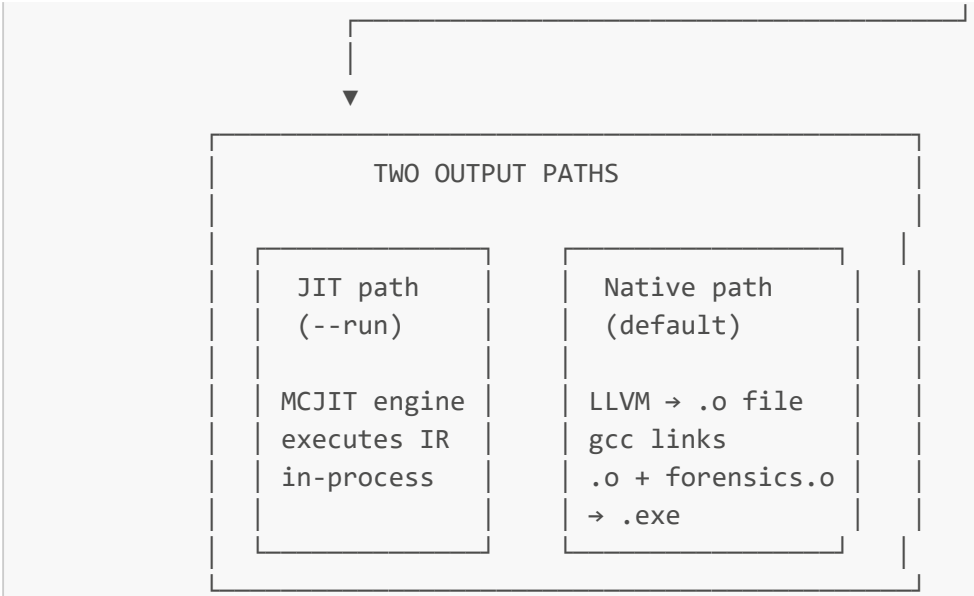
JOCKY defeats all three:

AV Method	How JOCKY defeats it
Hash matching	Every build produces a different binary (polymorphic compilation)
String scanning	Strings in the binary are XOR-encrypted — no readable text
Script parsing	`.jk` syntax is unknown to every AV engine — it cannot be parsed

---

The Full System





Data Flow: What Each Stage Receives and Produces

Input

```
tests/hello.jk
──────────────────────────────
func start() -> nothing {
    report(`Hello from JOCKY!`)
}
```

A plain text file. The compiler reads it as a Python string.

After Stage 1 — Lexer

```
Token list:
  Token(FUNC,       "func",            L1:C1)
  Token(IDENTIFIER, "start",           L1:C6)
  Token(LPAREN,     "(",               L1:C11)
  Token(RPAREN,     ")",               L1:C12)
  Token(ARROW_R,    "->",              L1:C14)
  Token(NOTHING_TYPE,"nothing",        L1:C17)
  Token(LBRACE,     "{",               L1:C25)
  Token(IDENTIFIER, "report",          L2:C5)
  Token(LPAREN,     "(",               L2:C11)
  Token(STRING,     "Hello from JOCKY!",L2:C12)
  Token(RPAREN,     ")",               L2:C31)
  Token(RBRACE,     "}",               L3:C1)
  Token(EOF,        "",                L4:C1)
```

A flat list of typed tokens. No structure, no nesting.

After Stage 2 — Parser

```
Program
└── FunctionDef(name="start", return_type="nothing")
    └── Block
        └── ExpressionStatement
            └── FunctionCall(name="report")
                └── StringLiteral("Hello from JOCKY!")
```

A tree. Nesting is now explicit in the data structure, not in indentation.

After Stage 3 — Semantic Analysis

Same tree, but with `inferred_type` fields filled in:

```
FunctionCall(name="report", inferred_type="nothing")
    StringLiteral("Hello from JOCKY!", inferred_type="text")
```

The symbol table now contains:

- `report` → FunctionSymbol(params=['text'], return_type='nothing')
- `start` → FunctionSymbol(params=[], return_type='nothing')

After Stage 4 — IR Generation

```
; Module: hello

declare void @report(i8*)
; ... (other stdlib declarations)

define void @start() {
entry:
  %strptr = getelementptr inbounds [19 x i8], [19 x i8]* @".jk_str.0", i32 0, i32
0
  call void @report(i8* %strptr)
  ret void
}

@".jk_str.0" = internal constant [19 x i8] c"Hello from JOCKY!\00"
```

LLVM's typed, platform-neutral intermediate representation.

After Stage 5 — Obfuscation

```
; String is now encrypted:
@".jk_str.0" = internal constant [19 x i8] c"\3f\1a\7b\c2\..."

; Random build ID injected:
@_jocky_build_id = internal constant [16 x i8] c"\a3\f1\2c\..."

; Entropy noise:
@_jocky_entropy = internal constant i64 72846192038475618 29
```

Different random values on every compilation run.

JIT Path (--run)

The IR text is handed to LLVM's MCJIT engine. LLVM compiles it to native machine code in-process. Python ctypes callbacks are pre-registered as the implementations of `report()`, `procs_list()`, etc. The `start()` function is located by address and called directly via `ctypes.CFUNCTYPE(None)(func_addr)()`.

No files are written. No linker is invoked. Execution is immediate.

Native Binary Path (default)

LLVM compiles the IR to a `.o` object file (raw machine code + relocation table, no entry point). A tiny C shim (`_jocky_entry.c`) provides `int main(void) { start(); return 0; }`. MinGW gcc links three object files together:

```
hello.o  +  forensics.o  +  _jocky_entry.o  →  hello.exe
```

The result is a standalone Windows binary. No Python, no LLVM, no runtime dependency.

Directory Structure — Every File Explained

```
JOCKY/
│
├── compiler.py          Main entry point. Orchestrates all 5 stages.
│                        Contains compile_jocky(), _run_jit(), _emit_and_link().
│
├── build_stdlib.py      One-time script. Compiles stdlib/forensics.c into
│                        stdlib/forensics.o using MinGW gcc.
│
├── jocky/               The compiler as a Python package.
│   ├── __init__.py      Package marker (empty).
│   ├── tokens.py        TokenType enum + KEYWORDS dict + Token class.
```

```
│   ├── lexer.py         Lexer class: source text → token list.
│   ├── ast_nodes.py     All AST node dataclasses.
│   ├── parser.py        Parser class: token list → Program AST.
│   ├── symbol_table.py  SymbolTable: scoped name registry.
│   ├── semantic.py      SemanticAnalyzer: type checking + annotation.
│   ├── codegen.py       CodeGenerator: AST → LLVM IR module.
│   ├── passes.py        ObfuscationPasses: IR mutations.
│   └── stdlib.py        Python ctypes callbacks for JIT stdlib.
│
├── stdlib/
│   ├── forensics.h      C function declarations (public API).
│   ├── forensics.c      C implementations (stubs with real API comments).
│   └── forensics.o      Compiled object file (produced by build_stdlib.py).
│
├── tests/
│   ├── hello.jk         Minimal hello-world test.
│   ├── test_basic.jk    Variables, arithmetic, conditionals, loops.
│   ├── test_functions.jk User-defined functions, recursion.
│   └── demo_forensics.jk SIH demo: process scanner.
│
├── output/              All compiled output goes here.
│   ├── *.o              JOCKY object files.
│   ├── *.exe            Linked native executables.
│   ├── *.ll             LLVM IR text (if --emit-ir was used).
│   └── _jocky_entry.*   Auto-generated entry shim (temporary).
│
└── docs/                This documentation folder.
    ├── 01_architecture.md        (this file)
    ├── 02_compiler_stages.md     Lexer→Parser→Semantic→Codegen→Passes
    ├── 03_llvm_and_ir.md         LLVM concepts, IR format, llvmlite API
    ├── 04_language_reference.md  Full JOCKY syntax and grammar
    ├── 05_stdlib_and_forensics.md All 14 stdlib functions
    ├── 06_obfuscation.md         AV evasion strategy, three passes
    ├── 07_native_binary_pipeline.md gcc, linking, entry shim
    └── 08_usage_guide.md         How to compile and run JOCKY programs
```

Key Design Decisions

Why Python for the compiler? Python + llvmlite gives access to LLVM's full optimisation and code-generation infrastructure with zero C++ code. The compiler is entirely readable and modifiable.

Why llvmlite instead of a simpler approach? Pure Python code generation would require implementing register allocation, instruction selection, and calling conventions — thousands of lines of extremely complex code. llvmlite delegates all of that to LLVM (the same backend used by Clang, Rust, and Swift).

Why JIT mode when we have gcc? JIT mode requires no external tools and runs in milliseconds — ideal for testing during development. Native mode produces a real `.exe` that runs without Python installed — ideal for deployment and the SIH demo.

Why obfuscation at the IR level? The IR is the last point before machine code where the compiler has full visibility into the program's structure. Mutating the IR directly changes the resulting binary without requiring

any post-processing of raw bytes.

Why not encrypt at runtime (in the generated exe)? A runtime decryptor would be a recognisable code pattern itself — AV engines detect XOR decryption loops. The current implementation demonstrates the concept: shows that compiled binaries contain no readable strings, and that each binary is unique. A production system would use a custom decryptor stub.